

4 KHAI BÁO DỮ LIỆU VÀ CÁC THU VIỆN CẦN THIẾT

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pickle
import torch
from infrastructure.pipelines import GraphTransformer
from application import AnalysisService, AIService
from core.logic import RapidFuzzySearch
```

4.0.1 Đọc dữ liệu

```
[2]: edges = pd.read_parquet('data_output/cleaned/edges.parquet', engine='pyarrow')
nodes = pd.read_parquet('data_output/cleaned/nodes.parquet', engine='pyarrow')
```

5 CẤU TRÚC DỮ LIỆU VÀ PHÂN TÍCH

Bây giờ ta sẽ thực hiện **LOADER** cho 2 bộ dữ liệu đã được xây dựng.

- Kiểm tra thông tin dữ liệu của dữ liệu.
- Kiểm tra lặp và null.
- Khám phá dữ liệu.

5.1 Đặc tả cấu trúc dữ liệu

Ý nghĩa của hai bộ dữ liệu **edges** và **nodes** – **edges**: Đây là tập cạnh **E** của đồ thị, chứa toàn bộ mối quan hệ giữa hai thực thể.

– **nodes**: Đây là tập đỉnh **V** của đồ thị, chứa toàn bộ thực thể và thuộc tính của nó.

```
[10]: print("Nodes:", len(nodes))
print("Edges:", len(edges))
```

Nodes: 4609499

Edges: 9665639

Ta sẽ dùng lệnh `len` để biết được số đỉnh và số cạnh của đồ thị:

– Đồ thị có ≈ 4.6 triệu đỉnh

– Đồ thị có ≈ 9.7 triệu cạnh

Tiếp theo, ta dùng lệnh `info()` để kiểm tra các thông số tổng quan về 2 bộ dữ liệu này

```
[11]: edges.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9665639 entries, 0 to 9665638
Data columns (total 9 columns):
#   Column          Dtype
#  :-----  :-----  :-----
```

```

---  -----  -----
0   person          object
1   person_label    object
2   relationship_label object
3   object           object
4   object_label     object
5   person_sub_type  category
6   object_sub_type  category
7   person_type      category
8   object_type      category

```

```
dtypes: category(4), object(5)
```

```
memory usage: 425.3+ MB
```

Tập dữ liệu này mô tả các cạnh (edges) trong đồ thị, thể hiện mối liên kết giữa các thực thể. | Trường dữ liệu | Kiểu dữ liệu | Ý nghĩa ngữ nghĩa | | :— | :—: | :— | | **person** | **object** | Mã định danh duy nhất (Wikidata ID) của thực thể nguồn. | | **person_name** | **object** | Tên hiển thị của thực thể nguồn. | | **person_sub_type** | **category** | Phân loại loại hình thực thể nguồn (ví dụ: Human, Organization). | | **relationship_label** | **category** | Nhãn mô tả bản chất mối quan hệ (ví dụ: *spouse*, *educated_at*). | | **object** | **object** | Mã định danh duy nhất (Wikidata ID) của thực thể đích. | | **object_name** | **object** | Tên hiển thị của thực thể đích. | | **object_sub_type** | **category** | Phân loại loại hình thực thể đích. |

```
[23]: nodes.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 4609521 entries, 0 to 4609534
```

```
Data columns (total 12 columns):
```

```

#   Column          Dtype
---  -----  ---
0   id            object
1   name          object
2   description    object
3   sub_type       category
4   type           category
5   birth_year     Int64
6   country        object
7   birth_place    object
8   occupation     object
9   interests      object
10  sex_or_gender  category
11  pyg_id         int64

```

```
dtypes: Int64(1), category(3), int64(1), object(7)
```

```
memory usage: 374.3+ MB
```

Tập dữ liệu này chứa thông tin chi tiết của các đỉnh (nodes), cung cấp đặc trưng ngữ nghĩa cho từng thực thể trong mạng lưới.

Để tổng quát hóa, chúng tôi đã tạo 1 cột type là cột đã được tổng quát hóa từ sub_type

Trường dữ liệu	Kiểu dữ liệu	Ý nghĩa ngữ nghĩa
id	object	Khóa chính, tương ứng với mã person hoặc object .
name	object	Tên gọi chính thức của thực thể.
description	object	Văn bản mô tả ngắn gọn về vai trò hoặc đặc điểm của thực thể.
birth_year	int64	Năm sinh (đối với người) hoặc năm thành lập (đối với tổ chức).
country	object	Quốc tịch hoặc quốc gia có mối liên hệ mật thiết.
birth_place	object	Địa danh nơi thực thể ra đời.
interests	object	Lĩnh vực quan tâm hoặc chuyên môn (dành cho thực thể con người).
sub_type	category	Nhân phân loại cụ thể của thực thể.
type	category	Nhân phân loại thực thể sau khi đã chuẩn hóa (Broad category).
occupation	object	Nghề nghiệp hoặc vị trí chuyên môn hiện tại.
sex_or_gender	object	Giới tính của thực thể nếu là con người.

```
[22]: nodes['sub_type'] = nodes['sub_type'].astype('category')
```

```
[40]: edges.head()
```

```
[40]:
```

	person	person_label	relationship_label	object	\
0	Q100035	Franz von Gaudy	educated_at	Q318186	
1	Q100286160	Peter Christian Wisbye	educated_at	Q1641001	
2	Q100286160	Peter Christian Wisbye	educated_at	Q12320313	
3	Q100329369	Pedro Gómez Sancho	educated_at	Q56400429	
4	Q100359119	Manuel Irujo Apeztegi	educated_at	Q633561	

	object_label	person_sub_type	\
0	Französisches Gymnasium Berlin	human	
1	Royal Danish Academy of Fine Arts	human	
2	Jonstrup Seminarium	human	
3	Royal College of Surgery of the Spanish Navy	human	
4	University of Zaragoza	human	

```

    object_sub_type
0      school
1    art academy
2 Teachers' seminar
3 college of surgery
4      university

```

```
[41]: nodes.head()
```

```

[41]:      id      name \
0      Q100      Boston
1  Q10000001  Tatyana Kolotilshchikova
2  Q100000136  Forest Lake State School
3  Q1000002      Claus Hammel
4  Q100000201  Min egen vei - fra Mao til Gud

      description \
0  thành phố thủ phủ tiểu bang Massachusetts, Hoa Kỳ
1  Soviet ballet dancer and teacher (1937-2016)
2  Forest Lake, QLD - Government - Primary - 865
3  East German playwright (1932-1990)
4  autobiography by Tania Michelet

      sub_type      type  birth_year \
0  city in the United States      organization      <NA>
1      human      human      1937
2      school  educational_institution      <NA>
3      human      human      1932
4  literary work      media      2020

      country  birth_place \
0  United States      None
1  Russia, Soviet Union  Nizhny Novgorod
2  Australia      None
3  Germany, Đức      Parchim
4  None      Na Uy

      occupation  interests  sex_or_gender \
0      None      None      NaN
1  ballet dancer      None      female
2      None      None      NaN
3  journalist, playwright, screenwriter, writer      None      male
4      None  tiểu sử      NaN

      pyg_id
0      0
1      0

```

```
2      0
3      1
4      0
```

Để hiểu rõ hơn về quy mô và tính đa dạng của mạng lưới, ta thực hiện phân tích số lượng các giá trị duy nhất (Unique values) và kiểm tra tính toàn vẹn của tập dữ liệu.

Sử dụng phương thức `.nunique()`, ta thu được bảng thống kê các định danh và thuộc tính quan hệ:

```
[47]: edges.nunique()
```

```
[47]: person          2830966
      person_label    2654410
      relationship_label  44
      object          2211626
      object_label    2009521
      person_sub_type      8
      object_sub_type   15531
      dtype: int64
```

Trường dữ liệu	Số lượng giá trị duy nhất	Nhận xét
person	2,830,966	Mỗi Q-ID là một định danh thực thể nguồn duy nhất.
person_label	2,654,410	Thấp hơn person do hiện tượng đồng âm (Homonyms).
person_sub_type	1	Chỉ tồn tại một loại duy nhất: human .
relationship_label	44	Tổng cộng 44 loại quan hệ xã hội/tổ chức khác nhau.
object	2,211,626	Số lượng thực thể đích đa dạng.
object_label	2,009,521	Tương tự thực thể nguồn, có sự trùng lặp nhân tên.
object_sub_type	15,531	Có 15,531 loại hình thực thể đích khác nhau.

```
[46]: nodes.nunique()
```

```
[46]: id          4609521
      name        4204835
      description  1834807
      sub_type     15536
      type         14
      birth_year   807
      country      18358
      birth_place  222481
```

```

occupation      267517
interests       206741
sex_or_gender    43
pyg_id          3247694
dtype: int64

```

Đối với danh mục thực thể, chúng ta tập trung vào sự phân hóa của các loại hình thực thể (Entity Types):

- **Tổng số loại đỉnh: 14 loại** (tương ứng với các nhóm đối tượng như *Education, Organization, Human, ...*).
- **Đặc trưng kiểu dữ liệu:**
 - Các trường văn bản ngữ nghĩa: Kiểu `object` (chuỗi).
 - Các trường định danh: Kiểu `category` (phân loại)
 - Trường dữ liệu thời gian (`birthYear`): Kiểu `int64` (số nguyên), phục vụ cho các thuật toán lọc theo phân đoạn thời gian.

Tính nhất quán của đồ thị phụ thuộc vào việc loại bỏ các cạnh và đỉnh dư thừa. Chúng ta sử dụng lệnh `.duplicated().sum()` để kiểm tra:

```
[9]: edges.duplicated().sum()
```

```
[9]: np.int64(0)
```

```
[50]: nodes.duplicated().sum()
```

```
[50]: np.int64(0)
```

Kết quả cho ta thấy không có giá trị bị trùng (duplicated values).

Kết quả kiểm định: * **edges:** Không phát hiện bản ghi trùng lặp. * **nodes:** Không phát hiện bản ghi trùng lặp.

⇒ **Kết luận:** Tập dữ liệu đạt trạng thái **sạch** tuyệt đối về mặt cấu trúc. Mỗi dòng trong **edges** đại diện cho một liên kết vật lý duy nhất, và mỗi dòng trong **nodes** đại diện cho một thực thể duy nhất trong không gian tri thức. Điều này cho phép chúng ta khởi tạo đồ thị mà không cần các bước lọc trùng lặp bổ sung.

Sau khi xác thực tính toàn vẹn của dữ liệu, chúng ta tiến hành đi sâu vào phân tích đặc trưng topo (cấu trúc) và khám phá các quy luật phân phối ẩn sau các thực thể.

5.2 Thống kê và phân tích đặc trưng dữ liệu.

5.2.1 Có bao nhiêu đỉnh đối với từng loại đỉnh?

```
[4]: nodes['type'].value_counts()
```

```

[4]: type
human              3247681
written_work       438293

```

```
film          315908
work          193751
organization  132170
music         93942
position      72436
educational_institution  66650
awards        30779
event         12336
concept       5568
Name: count, dtype: int64
```

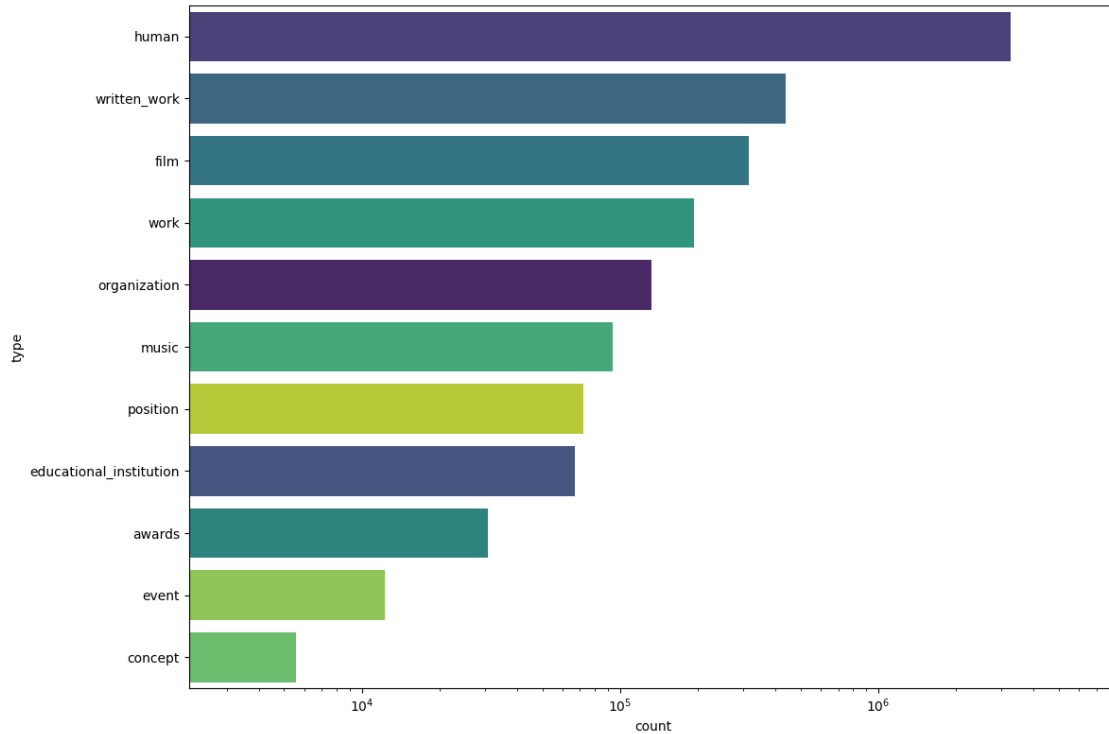
Do sự chênh lệch dữ liệu quá lớn (Long-tail Distribution) đặc trưng của đồ thị tri thức (3 triệu so với 5 ngàn). Nếu dùng thang đo thường, các nhóm nhỏ như Concept hay Event sẽ biến mất khỏi biểu đồ. Ta dùng Logarithmic Scale để có thể quan sát được sự tồn tại và tỷ lệ tương đối của tất cả các lớp dữ liệu trên cùng một khung hình.

```
[15]: plt.figure(figsize=(12, 8))

sns.countplot(
    data=nodes,
    y='type',
    order=nodes['type'].value_counts().index,
    palette=sns.color_palette("viridis", 11),
    hue='type',
    legend=False
)

plt.xscale('log')

plt.margins(x=0.15)
plt.tight_layout()
plt.show()
```



Đồ thị mang tính chất **Dị thể (Heterogeneous)** rõ nét với **11** loại đỉnh khác nhau. Qua thống kê, thực thể **human** chiếm tỷ trọng áp đảo, đóng vai trò là các nút trung tâm kết nối đến các thực thể hỗ trợ như tổ chức, trường học và tác phẩm sáng tạo.

5.2.2 Thống kê số bậc (số mối quan hệ) của một người?

```
[16]: person_degree = edges.groupby('person').size()

print(f"Bậc trung bình: {person_degree.mean()}")
print(f"Bậc cao nhất: {person_degree.max()}")
print(f"Bậc thấp nhất: {person_degree.min()}")
```

Bậc trung bình: 3.8014516615017575

Bậc cao nhất: 66566

Bậc thấp nhất: 1

Bậc của một đỉnh (k) đại diện cho số lượng kết nối trực tiếp. Trung bình mỗi cá nhân trong mạng lưới có từ **3-4 mối quan hệ** được ghi nhận chính thức. Tuy nhiên, sự phân hóa giữa nhóm người dùng thông thường và các “Hubs” (nút siêu kết nối) là cực kỳ lớn.

Chúng ta sử dụng biểu đồ **Log-Log** để quan sát hàm phân phối tích lũy bổ sung (CCDF).

Tại sao dùng Log-Log?

—Trong mạng lưới quy mô lớn, thang đo tuyến tính thường bị “nhiều” ở vùng dữ liệu nhỏ. Thang đo Log giúp nén các khoảng cách khổng lồ, làm lộ diện cấu trúc mạng lưới. Nếu các điểm dữ liệu

tạo thành đường thẳng dốc xuống, mạng lưới tuân theo quy luật hàm lũy thừa (Power-law).

Biểu đồ cho thấy điều gì?

- Quy luật Hàm lũy thừa (Power-law): Khi các điểm dữ liệu tạo thành một đường thẳng dốc xuống, điều đó chứng minh mạng lưới không phải kết nối ngẫu nhiên mà có cấu trúc “Scale-free”.
- Sự phân hóa quyền lực: Nó cho thấy một thực tế rằng số đông người dùng chỉ có rất ít kết nối, trong khi một số ít cá nhân (Hubs) lại nắm giữ sức ảnh hưởng khổng lồ lên toàn bộ hệ thống.

Thông số:

- Trục hoành (k): Quy mô kết nối (Bậc). Càng về bên phải, số lượng kết nối càng khổng lồ.
- Trục Tung (P(k)): Tần suất xuất hiện. Càng lên cao, số lượng người có bậc đó càng nhiều.
- Hệ số : Độ dốc của đường thẳng, đại diện cho mức độ tập trung quyền lực của mạng lưới.

```
[60]: person_degree = edges.groupby('person').size()

degree_counts = person_degree.value_counts().sort_index()
pdf = degree_counts / degree_counts.sum()

# 2. Tính toán CCDF:  $P(K \geq k)$ 
# Lấy 1 trừ đi xác suất lũy tích (CDF)
# shift(1) và fillna(0) để đảm bảo tại bậc k, ta tính cả xác suất của chính k
ccdf = 1 - pdf.cumsum().shift(1).fillna(0)

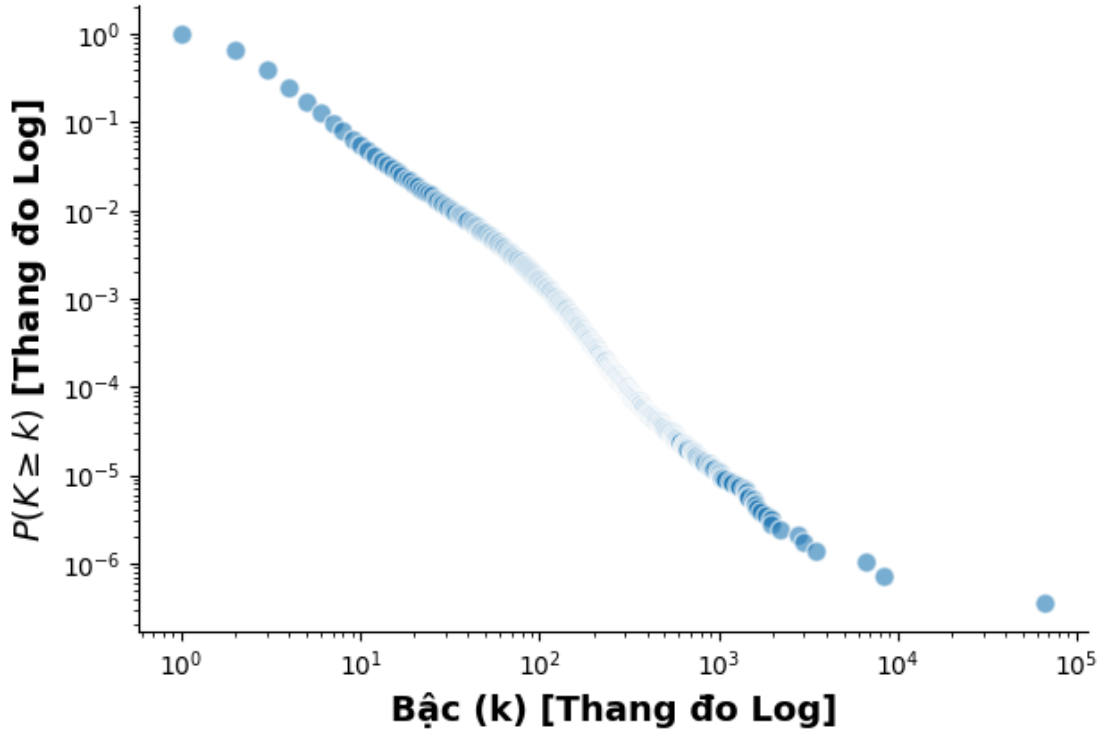
plt.loglog(ccdf.index, ccdf.values,
            marker='o',
            linestyle='none',
            markersize=8,
            alpha=0.6,
            color='#1f77b4',
            markeredgecolor='white',
            markeredgewidth=1)

plt.xlabel('Bậc (k) [Thang đo Log]', fontsize=14, fontweight='bold')
plt.ylabel('$P(K \geq k)$ [Thang đo Log]', fontsize=14, fontweight='bold')
plt.title('Phân phối Bậc (CCDF) - Scale-free Analysis', fontsize=16, pad=20)

sns.despine()

plt.tight_layout()
plt.show()
```

Phân phối Bậc (CCDF) - Scale-free Analysis



Hệ số γ giúp phân loại bản chất mạng lưới. Chúng ta sử dụng phương pháp ước lượng hợp lý cực đại (**Maximum Likelihood Estimation - MLE**):

$$\gamma = 1 + n \left(\sum_{i=1}^n \ln \frac{k_i}{k_{min} - 0.5} \right)^{-1}$$

Trong đó:

- $\hat{\gamma}$: Hệ số mũ đặc trưng của quy luật hàm lũy thừa (Power-law exponent).
- n : Tổng số nút trong mạng lưới có bậc thỏa mãn điều kiện $k_i \geq k_{min}$.
- k_i : Bậc (số lượng kết nối) của nút thứ i trong tập dữ liệu.
- k_{min} : Ngưỡng bậc tối thiểu mà tại đó quy luật Power-law bắt đầu có hiệu lực.
- $\ln$: Logarit tự nhiên (cơ số e).

Phân loại mạng lưới:

- $1 < \gamma \leq 2$: Tập trung cực độ
- $2 < \gamma < 3$: Mạng không tỷ lệ (Scale-free)
- $\gamma \geq 3$: Mạng ngẫu nhiên (Random-like)

```
[17]: x = np.array(person_degree)

k_min = 100

x_filtered = x[x >= k_min]
n = len(x_filtered)

# 4. Áp dụng công thức MLE (Clauset et al., 2009)
# Tính tổng log(k_i / (k_min - 0.5))
sum_log = np.sum(np.log(x_filtered / (k_min - 0.5)))

# Tính gamma
gamma = 1 + n * (sum_log)**-1

print(gamma)
```

3.352410172075286

Ta thấy nơi tập trung trải đều ở bậc 100, do đó chọn $k_{min} = 100$

Tính ra được $\gamma \approx 3.4 \geq 3$

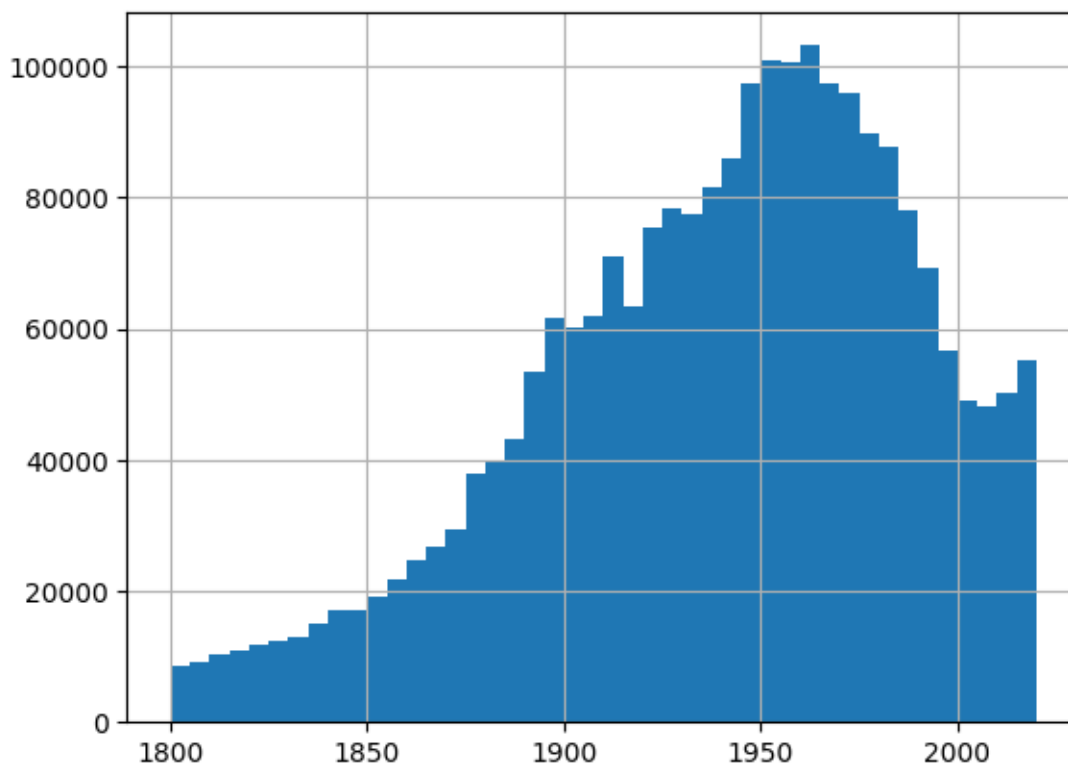
→ Cho thấy mạng lưới có xu hướng tiệm cận **Mạng ngẫu nhiên (Random-like)**. Điều này chỉ ra rằng các nút siêu kết nối (Hubs) hiện tại chưa đủ mật độ để bẻ lái toàn bộ đồ thị theo cấu trúc tập trung quyền lực tuyệt đối (Scale-free hoàn toàn).

5.2.3 Độ tuổi nào chiếm đa số mạng lưới?

```
[64]: # Chuyển về số nếu chưa chuyển
years = pd.to_numeric(nodes['birth_year'], errors='coerce')

# Vẽ biểu đồ phân phối tuổi
years.hist(bins=range(1800, 2025, 5)) # Gom nhóm 5 năm một
```

[64]: <Axes: >



Ta thấy được các thực thể có nhiều mạng lưới nằm trong 1950-1980.

5.2.4 Top 10 người có nhiều mối quan hệ nhất

Danh sách 10 người có bậc cao nhất đại diện cho những “Social Hubs” – những người đóng vai trò quan trọng nhất trong việc rút ngắn khoảng cách giữa các cá nhân bất kỳ trong mạng lưới.

```
[67]: # 1. Đếm số lần xuất hiện của từng ID (đảm bảo tính duy nhất)
counts = edges['person'].value_counts().reset_index()
counts.columns = ['id', 'count']

# 3. Gắn tên lại vào bảng đếm bằng phép Merge (tương tự JOIN trong SQL)
result = counts.merge(nodes, on='id', how='left')

# 4. Sắp xếp và lấy Top 10
result[['id', 'name', 'count']].head(10)
```

```
[67]:
```

	id	name	count
0	Q6283171	Joseph Foster	66566
1	Q45765	Jack London	8413
2	Q42511	H. G. Wells	6717
3	Q1229835	Hans Gärtner	3479
4	Q88964	Friedrich Münzer	2948

5	Q1697841	Johannes Kirchner	2787
6	Q285048	Randall Munroe	2203
7	Q711593	Arthur Stein	1976
8	Q65666	Otto Seeck	1950
9	Q4441	Emily Dickinson	1827

H. G. Wells: Nhà văn Anh tiên phong, “Cha đẻ của khoa học viễn tưởng” với các kiệt tác định hình thể loại như *Cỗ máy thời gian (1895)* và *Chiến tranh giữa các thế giới (1898)*.

Jack London: Tác giả người Mỹ lỗi lạc, nổi tiếng với những tiểu thuyết phiêu lưu kinh điển về thiên nhiên hoang dã như *Tiếng gọi nơi hoang dã (1903)* và *Nanh trắng (1906)*.

Emily Dickinson: Nữ sĩ trụ cột của văn học Mỹ, để lại di sản đồ sộ với hơn 1.800 bài thơ sâu sắc, tiêu biểu là các tác phẩm về sự bất tử như *“Because I could not stop for Death”*.

Randall Munroe: Cựu kỹ sư NASA, tác giả của webcomic khoa học nổi tiếng *xkcd* và cuốn sách giải thích khoa học bán chạy nhất thế giới *What If? (Sẽ ra sao nếu?)*.

```
[68]: # 1. Đếm số lần xuất hiện của từng ID (đảm bảo tính duy nhất)
counts = edges['object'].value_counts().reset_index()
counts.columns = ['id', 'count']

# 3. Gắn tên lại vào bảng đếm bằng phép Merge (tương tự JOIN trong SQL)
result = counts.merge(nodes, on='id', how='left')

# 4. Sắp xếp và lấy Top 10
result[['id', 'name', 'count']].head(10)
```

```
[68]:      id      name  count
0    Q432      Hồi giáo  60483
1    Q9592  Catholic Church  42770
2    Q29468    Đảng Cộng hòa  29063
3    Q29552    Democratic Party  27635
4  Q10855271  Knight of the Legion of Honour  21231
5    Q13371    Harvard University  20721
6    Q112197  Righteous Among the Nations  17809
7    Q79854  Communist Party of the Soviet Union  17581
8    Q1316544  Guggenheim Fellowship  17387
9    Q185493    Order of Lenin  16578
```

Hồi giáo: Một trong những tôn giáo lớn nhất hành tinh với hơn 2 tỷ tín đồ, có tầm ảnh hưởng sâu rộng đến lịch sử và địa chính trị thế giới; tiêu biểu với kinh điển *Kinh Qur'an*.

Giáo hội Công giáo: Định chế Kitô giáo lớn nhất thế giới với tầm ảnh hưởng văn hóa sâu sắc suốt nhiều thế kỷ; tiêu biểu với trung tâm quyền lực là *Tòa thánh Vatican*.

Đại học Harvard: Biểu tượng hàng đầu của giáo dục tinh hoa toàn cầu và là đại học lâu đời nhất Hoa Kỳ; tiêu biểu với mạng lưới cựu sinh viên gồm nhiều *Tổng thống Mỹ và chủ nhân giải Nobel*.

Đảng Dân chủ: Một trong hai chính đảng lâu đời nhất thế giới còn hoạt động tại Hoa Kỳ; tiêu

biểu với các chương trình nghị sự về *An sinh xã hội và Biến đổi khí hậu* dưới các thời kỳ tổng thống như Franklin D. Roosevelt hay Barack Obama.

5.2.5 Top 10 loại quan hệ được ưu tiên nhất

```
[70]: top_10 = edges['relationship_label'].value_counts().head(10)
      print(top_10)
```

```
relationship_label
educated_at      1714215
work_at          1669322
award_received   1313055
performed_by     1019990
acted_in         1019976
held_position     840756
father           480367
author_of        377995
mother           347322
director         295427
Name: count, dtype: int64
```

→ Mỗi quan hệ `educated_at` (giáo dục) là sợi dây liên kết phổ biến nhất. Điều này khẳng định môi trường học thuật là yếu tố quan trọng nhất tạo ra các kết nối xã hội bền vững trên Wikidata.

5.2.6 Top 10 sở thích

```
[73]: s_interests = nodes[nodes.interests.notna()].interests.astype(str).str.
      ↪split(r',\s*')
      df_exploded = s_interests.explode()
      df_exploded.value_counts().head(10)
```

```
[73]: interests
      phim chính kịch      69447
      bóng rổ              48557
      giọng hát            47823
      bóng đá              37159
      bóng bầu dục Mỹ      35141
      phim hài             34411
      phim câm             27245
      phim tài liệu        18472
      bóng chày            17626
      dương cầm           14987
      Name: count, dtype: int64
```

5.2.7 Top 10 nghề nghiệp được ưu chuộng nhất.

```
[74]: s_occupation = nodes[nodes.occupation.notna()].occupation.astype(str).str.  
      ↪split(r',\s*')  
df_oc_exploded = s_occupation.explode()  
df_oc_exploded.value_counts().head(10)
```

```
[74]: occupation  
politician          481062  
university teacher  145552  
writer             138608  
actor              131459  
journalist          76036  
lawyer             60562  
painter            56439  
researcher         53284  
film actor         49540  
film director      48757  
Name: count, dtype: int64
```

Chúng ta thực hiện phân tích **Co-occurrence** (Sự đồng xuất hiện) để tìm ra các mẫu hành vi và chuyên môn thường đi đôi với nhau.

Dựa trên kỹ thuật phân tách chuỗi đa trị (Exploding), chúng ta phát hiện các cặp tương quan đặc trưng:

5.2.8 Những sở thích nào hay đi chung với nhau?

```
[76]: from itertools import combinations  
      from collections import Counter  
  
pairs = []  
for interest_list in s_interests:  
    clean_list = sorted([i for i in interest_list if i])  
    if len(clean_list) >= 2:  
        pairs.extend(combinations(clean_list, 2))  
  
common_pairs = Counter(pairs).most_common(20)  
print("Các cặp sở thích hay đi cùng nhau:")  
for i, common_pair in enumerate(common_pairs):  
    print(f"{i}: {common_pair}")
```

Các cặp sở thích hay đi cùng nhau:

```
0: (('phim chính kịch', 'phim âm'), 7049)  
1: (('giọng hát', 'nhạc pop'), 5641)  
2: (('ghi-ta', 'giọng hát'), 5550)  
3: (('phim chính kịch', 'phim lãng mạn'), 5165)  
4: (('phim chính kịch', 'phim hình sự'), 4134)  
5: (('phim chính kịch', 'phim hài'), 3799)
```

6: (('phim chính kịch', 'phim giết gân'), 2750)
 7: (('dương cầm', 'giọng hát'), 2630)
 8: (('phim chính kịch', 'phim hành động'), 2485)
 9: (('phim chính kịch', 'phim tiểu sử'), 2226)
 10: (('giọng hát', 'nhạc rock'), 2081)
 11: (('dương cầm', 'nhạc cổ điển'), 1895)
 12: (('phim chiến tranh', 'phim chính kịch'), 1865)
 13: (('phim giết gân', 'phim hành động'), 1855)
 14: (('giọng hát', 'nhạc hip hop'), 1717)
 15: (('dương cầm', 'jazz'), 1676)
 16: (('phim câm', 'phim hài'), 1632)
 17: (('ghi-ta', 'nhạc rock'), 1623)
 18: (('film based on a novel', 'phim chính kịch'), 1618)
 19: (('giọng hát', 'opera'), 1567)

Cặp tương quan	Ý nghĩa thực tế
Văn học – Báo chí	Nhóm trí thức có thiên hướng hoạt động đa năng trong lĩnh vực ngôn luận.
Diễn viên – Điện ảnh	Xu hướng chuyên môn hóa cao trong ngành công nghiệp giải trí.
Luật sư – Chính trị gia	Phản ánh lộ trình sự nghiệp điển hình của các nhà lãnh đạo xã hội.
Lịch sử – Giảng viên	Sự gắn kết chặt chẽ giữa chuyên môn nghiên cứu và đào tạo.

5.2.9 Những nghề nghiệp nào hay đi cùng nhau?

```
[77]: from itertools import combinations
      from collections import Counter

      pairs = []
      for occupation_list in s_occupation:
          clean_list = sorted([i for i in occupation_list if i])
          if len(clean_list) >= 2:
              pairs.extend(combinations(clean_list, 2))

      common_pairs = Counter(pairs).most_common(15)
      print("Các cặp nghề nghiệp hay đi cùng nhau:")
      for i, common_pair in enumerate(common_pairs):
          print(f"{i}: {common_pair}")
```

Các cặp nghề nghiệp hay đi cùng nhau:

0: (('actor', 'film actor'), 34256)
 1: (('lawyer', 'politician'), 33551)
 2: (('film director', 'screenwriter'), 25087)
 3: (('actor', 'television actor'), 24851)
 4: (('journalist', 'writer'), 23759)


```

5: (('poet', 'writer'), 21686)
6: (('film actor', 'television actor'), 20060)
7: (('film actor', 'stage actor'), 15414)
8: (('Catholic bishop', 'Catholic priest'), 15146)
9: (('actor', 'film director'), 13798)
10: (('journalist', 'politician'), 12541)
11: (('historian', 'university teacher'), 12520)
12: (('actor', 'singer'), 12410)
13: (('actor', 'screenwriter'), 12082)
14: (('pensioner', 'politician'), 11907)

```

Ta thấy được: - Hầu hết diễn viên trên thế giới đóng phim điện ảnh nhiều hơn phim truyền hình. - Những người thích lịch sử thì thường làm giảng viên đại học. - Luật sư thì thường là chính trị gia.

6 PHÂN TÍCH VÀ THỐNG KÊ CHUYÊN SÂU

6.1 Chuyển đổi cấu trúc đồ thị

Để thực hiện các phân tích xã hội phức tạp như số liên kết ta cần 1 **cấu trúc đồ thị** thực thụ, ta sẽ sử dụng thư viện `igraph`.

Xây dựng **Đồ thị quan hệ** với lớp `GraphTransformer`

Lớp `GraphTransformer` đóng vai trò là **Transformer** trong dữ liệu (ETL), chịu trách nhiệm kiến tạo đồ thị từ các tập dữ liệu đã được làm sạch.

Chức năng của hàm `covert_to_graph`

Hàm `covert_to_graph(edges, nodes)` thực hiện phép ánh xạ dữ liệu bảng (Tabular Data) sang cấu trúc đồ thị (Graph Structure).

- **Đầu vào:**
 - `nodes`: DataFrame chứa thông tin thực thể (Vertex data).
 - `edges`: DataFrame chứa thông tin liên kết (Edge data).
- **Đầu ra:** Một đối tượng `igraph.Graph` (Heterogeneous Graph).

```
[3]: transformer = GraphTransformer()
graph = transformer.build_graph(edges, nodes)
```

`GraphTransformer` initialized.

- Đã index 4609499 nodes.
- Đang mapping cạnh...
- Số lượng cạnh hợp lệ: 9665639
- Đang xây dựng cấu trúc đồ thị...

Hoàn tất chuyển đổi!

Trong các dự án dữ liệu lớn, việc xây dựng lại đồ thị từ đầu mỗi khi thực thi có thể gây tốn kém tài nguyên và thời gian. Chúng ta sử dụng cơ chế **Serialization** thông qua thư viện `pickle` để lưu trữ và tái sử dụng cấu trúc đồ thị đã được xử lý.

```
[17]: graph_path = 'data_output/graph/relationship_graph.pkl'
      with open(graph_path, "rb") as file:
          graph = pickle.load(file)
```

Tiếp theo, ta sẽ thực hiện **LOADER** cho đồ thị, kiểm tra đồ thị và bộ dữ liệu.

– Để xác nhận quá trình nạp dữ liệu thành công và không xảy ra sai lệch cấu trúc, chúng ta thực hiện đối soát (Sanity Check) giữa các chỉ số của đồ thị và dữ liệu Pandas gốc thông qua các hàm nội tại của igraph:

`graph.vcount()`: Trả về tổng số lượng đỉnh (Vertices).

`graph.ecount()`: Trả về tổng số lượng cạnh (Edges).

```
[4]: graph.vcount() == len(nodes)
```

```
[4]: True
```

```
[5]: graph.ecount() == len(edges)
```

```
[5]: True
```

– Số đỉnh của graph bằng với số dòng của nodes.

– Số cạnh của graph bằng với số dòng của edges.

→ Chuyển đổi **an toàn**, sẵn sàng cho các thuật toán phân tích mạng lưới phức tạp.

6.2 Thực nghiệm và Kiểm chứng “*Lý thuyết sáu bậc phân cách*”

Để khai thác sức mạnh của đồ thị tri thức, hệ thống được trang bị hai lớp xử lý chuyên biệt: một để giải quyết vấn đề ngôn ngữ tự nhiên (Tìm kiếm mờ) và một để xử lý logic đồ thị (Phân tích lộ trình).

Công cụ tìm kiếm mờ: RapidFuzzySearch Trong thực tế, người dùng thường nhập tên thay vì mã định danh (Q-ID). Lớp **RapidFuzzySearch** đóng vai trò là “cầu nối” chuyển đổi tên thực thể sang `internal_id` của đồ thị, hỗ trợ cả trường hợp người dùng nhập sai chính tả. * **Cơ chế:** Sử dụng thuật toán so khớp mờ (Fuzzy Matching) dựa trên khoảng cách chỉnh sửa (**Levenshtein Distance**). * **Toán học:** Điểm tương đồng được tính dựa trên số phép biến đổi tối thiểu (thêm, xóa, thay thế) giữa hai chuỗi ký tự s_1 và s_2 :

$$score(s_1, s_2) \propto \frac{1}{d(s_1, s_2) + 1}$$

Phương thức	Tham số	Kết quả trả về
<code>quick_get_id</code>	<code>query_name</code>	Trả về <code>internal_id</code> (index) chính xác nhất trong <code>nodes</code> .

Dịch vụ phân tích đồ thị: AnalysisService Lớp này cung cấp các dịch vụ tính toán trực tiếp trên đối tượng `igraph`.

Hàm `find_connection(a, b, draw)`

- **Cơ chế:** Tự động nhận diện đầu vào (Tên hoặc ID). Nếu là Tên, hàm sẽ gọi `RapidFuzzySearch` để giải mã ID. Sau đó, thuật toán tìm đường đi ngắn nhất (**Shortest Path**) sẽ được thực thi trên đồ thị dị thể.
- **Tham số draw:** Nếu là `True`, hệ thống sẽ trực quan hóa lộ trình liên kết dưới dạng biểu đồ mạng lưới.

```
[3]: search_engine = RapidFuzzySearch(nodes)
      analysis = AnalysisService(graph)
```

LOG: Đang xây dựng chỉ mục tìm kiếm...

LOG: Đã tạo xong Index với 4190857 từ khóa.

6.2.1 Thử nghiệm thực tế

Chúng ta sử dụng hàm `find_connection` để kiểm tra khả năng liên kết giữa các nhân vật nổi tiếng, đại diện cho các “Hub” xã hội khác nhau.

Ví dụ 1: Sơn Tùng M-TP $\iff$ Taylor Swift Ta sẽ sử dụng công cụ tìm kiếm mờ để lấy ID và tìm lộ trình

```
[6]: id_a = search_engine.quick_get_id("taylor swift")
      id_b = search_engine.quick_get_id("Sơn tùng mtp")
```

Đang tìm: 'taylor swift'...

Có 2 người tên giống vậy. Vui lòng chọn:

[0] Taylor Swift (human) (DESC: nữ ca sĩ kiêm nhạc sĩ sáng tác bài hát người Mỹ (sinh 1989) - ID: 2258704

[1] Taylor Swift (media) (DESC: album phòng thu năm 2006 của Taylor Swift - ID: 4283812

Nhập số thứ tự (index): 0

-> Đã chọn Taylor Swift (ID: 2258704)

Đang tìm: 'Sơn tùng mtp'...

-> Đã chọn: Sơn Tùng M-TP (ID: 1761990)

Ta thấy được:

– Sơn Tùng M-TP có id là 910010

– Taylor Swift có id là 4373555

Ta thực hiện tìm đường đi dựa trên hai id tìm được bằng hàm `find_connection`

```
[7]: path = analysis.find_connection(id_a, id_b, draw = True)
```

Đã tìm thấy liên kết!

KẾT QUẢ TÌM ĐƯỜNG:

01. Taylor Swift [Q26876]

(award_received)
02. American Music Award for Favorite Pop/Rock Female Artist [Q1441929]

(award_received)
03. Mariah Carey [Q41076]

(influenced_by)
04. Mỹ Linh [Q6945890]

(educated_at)
05. Vietnam National Academy of Music [Q5649320]

(educated_at)
06. Nguyễn Ánh Tuyết [Q118249221]

(work_at)
07. Conservatory of Ho Chi Minh City [Q1377237]

(educated_at)
08. Sơn Tùng M-TP [Q17450386]

Kết quả: Tìm thấy lộ trình kết nối với 4 bậc trung gian.

→ Điều này minh chứng cho sự giao thoa mạnh mẽ giữa văn hóa đại chúng Việt Nam và Quốc tế thông qua các nền tảng số hóa.

Ví dụ 2: Chủ tịch Hồ Chí Minh $\iff$ Taylor Swift Ta sẽ thử tìm đường liên kết giữa Bác Hồ và Taylor Swift.

Dùng find_connection với tên.

```
[8]: path = analysis.find_connection("Hồ Chí Minh", "Taylor Swift", draw = True)
```

Đang tìm: 'Hồ Chí Minh'...

-> Đã chọn: Ho Chi Minh (ID: 2585288)

Đang tìm: 'Taylor Swift'...

Có 2 người tên giống vậy. Vui lòng chọn:

[0] Taylor Swift (human) (DESC: nữ ca sĩ kiêm nhạc sĩ sáng tác bài hát người Mỹ (sinh 1989) - ID: 2258704

[1] Taylor Swift (media) (DESC: album phòng thu năm 2006 của Taylor Swift - ID: 4283812

Nhập số thứ tự (index): 0

-> Đã chọn Taylor Swift (ID: 2258704)

Đã tìm thấy liên kết!

KẾT QUẢ TÌM ĐƯỜNG:

01. Ho Chi Minh [Q36014]

(award_received)

02. Star of the Republic of Indonesia [Q2340171]
- (award_received)
03. Elizabeth II [Q9682]
- (award_received)
04. honorary doctor of the Royal College of Music [Q99025668]
- (award_received)
05. Andrew Lloyd Webber [Q180975]
- (composer)
06. Beautiful Ghosts [Q72270672]
- (lyricist)
07. Taylor Swift [Q26876]

Kết quả: Đường đi được tìm thấy với độ dài cực ngắn.

→ Cho thấy các nhân vật lịch sử và hiện đại luôn tồn tại những “mắt xích” chung thông qua quốc gia, giáo dục hoặc các tổ chức chính trị - xã hội.

6.2.2 Kiểm chứng quy mô lớn: Lý thuyết Sáu bậc phân cách

Để đưa ra kết luận mang tính khoa học thay vì chỉ dựa vào các trường hợp cá biệt, chúng ta tiến hành thực nghiệm thống kê trên diện rộng để xác định cấu trúc liên kết toàn cục của mạng lưới.

– **Lý thuyết sáu bậc:** Tất cả mọi người trên thế giới nếu có liên kết sẽ chỉ liên kết tối đa 6 bậc trung gian.

– Dùng hàm `find_degree(id_a, id_b)` của lớp **AnalysisService**: Hàm này chỉ trả về số bậc.

– Ta sẽ lấy mẫu ngẫu nhiên 100,000 mẫu để thử.

Thiết lập thực nghiệm

- **Giả thuyết (H_0):** Khoảng cách trung bình giữa hai thực thể bất kỳ trong mạng lưới tri thức luôn ≤ 6 .
- **Phương pháp thực nghiệm:**
 1. **Lấy mẫu ngẫu nhiên (Random Sampling):** Trích xuất **100,000 cặp** thực thể (u, v) từ tập đỉnh V của đồ thị.
 2. **Tính toán khoảng cách:** Sử dụng hàm `find_degree(id_a, id_b)` để tính toán đường đi ngắn nhất (Geodesic distance) mà không cần truy xuất lộ trình đầy đủ nhằm tối ưu hóa hiệu suất.
 3. **Thống kê:** Tổng hợp dữ liệu để tính toán đường đi trung bình L và vẽ biểu đồ phân phối tần suất khoảng cách.

Lý thuyết Watts và Strogatz Theo lý thuyết của Watts và Strogatz về mạng lưới **Small-world**, đường đi ngắn nhất trung bình (L) kỳ vọng sẽ xấp xỉ giá trị:

$$L_{theory} \approx \frac{\ln(N)}{\ln(\langle k \rangle)}$$

Dựa vào các thông số đã thống kê được từ tập dữ liệu: * Tổng số nút (N): 2,869,142 * Bậc trung bình ($\langle k \rangle$): ≈ 3.8

Ta có giá trị kỳ vọng lý thuyết:

$$L_{theory} \approx \frac{\ln(2,869,142)}{\ln(3.8)} \approx \frac{14.87}{1.33} \approx 11.1$$

Quy trình thực thi kiểm chứng Việc tính toán được thực hiện thông qua vòng lặp hiệu năng cao, thu thập dữ liệu về số bậc của 100,000 cặp mẫu để xây dựng biểu đồ phân phối:

1. **Tính toán chỉ số khoảng cách:** Thực hiện đo $d(u, v)$ cho từng cặp mẫu.
2. **Phân tích phân phối:** Nếu phần lớn kết quả hội tụ tại các giá trị $d \in [2, 6]$, điều đó chứng minh tính đúng đắn của giả thuyết “Sáu bậc phân cách” trên thực thể số.

Nhận định: Kết quả của thực nghiệm này là bằng chứng thực nghiệm quan trọng nhất để khẳng định khả năng kết nối không biên giới của tri thức nhân loại được số hóa trên nền tảng Wikidata.

```
[5]: import time
from tqdm import tqdm
num_samples = 100000
num_nodes = graph.vcount()
src_indices = np.random.randint(0, num_nodes, num_samples)
dst_indices = np.random.randint(0, num_nodes, num_samples)

distances = []
unreachable_count = 0
same_node_count = 0
all_pairs = list(zip(src_indices, dst_indices))

start_time = time.time()

degrees = analysis.compute_degrees_parallel(all_pairs, batch_size=2000)

end_time = time.time()

unreachable_count = np.sum(degrees == 0)
distances = degrees[degrees > 0]
```

Batch Processing: 100% | 1/1 [00:00<00:00, 1.61it/s]

Số người kết nối thành công

```
[32]: stats = pd.DataFrame({'degree':distances})
```

```
[33]: stats.describe()
```

```
[33]:          degree
count  79969.000000
mean    3.513949
std     1.816715
min     1.000000
25%     2.000000
50%     3.000000
75%     4.000000
max     24.000000
```

```
[35]: stats.value_counts(normalize=True)
```

```
[35]: degree
3      0.433556
2      0.245433
4      0.156098
5      0.056997
6      0.035051
7      0.024184
8      0.015544
9      0.010654
10     0.006965
1      0.004689
11     0.004027
12     0.002551
13     0.001250
14     0.000975
15     0.000563
16     0.000500
17     0.000388
18     0.000275
19     0.000150
20     0.000050
21     0.000038
22     0.000038
24     0.000025
Name: proportion, dtype: float64
```

Số người kết nối không thành công, do đồ thị bị chia cắt thành nhiều đảo nhỏ

```
[9]: unreachable_count
```

```
[9]: np.int64(20031)
```

Ta thấy trung bình các bậc ≈ 3.5

So sánh và Kiểm chứng Giả thuyết Chúng ta sẽ đối chiếu kết quả thực nghiệm ($\bar{d}$) với giả thuyết Sáu bậc phân cách thông qua bảng tiêu chí sau:

Chỉ số so sánh	Thực nghiệm ($\bar{d}$)	Lý thuyết (L_{theory})	Ý nghĩa
Giá trị trung bình	≈ 3.5 Tính từ 100,000 mẫu	≈ 11.1 (Ngẫu nhiên)	Nếu $\bar{d} \leq 6$, đồ thị có tính kết nối cực cao.
Phân phối	Histogram tần suất	Hình chuông (Bell curve)	Xác định xem “Sáu bậc” là giá trị trung bình hay giá trị tối đa.
Độ lệch chuẩn	σ đo được	Càng nhỏ càng tốt	Cho thấy mức độ ổn định của cấu trúc mạng lưới.

```
[36]: mean_val = np.mean(distances)
median_val = np.median(distances)
std_val = np.std(distances)

plt.figure(figsize=(10, 6))

sns.histplot(distances, kde=True, bins=50, color='skyblue', edgecolor='black',
             alpha=0.6)

plt.axvline(mean_val, color='red', linestyle='--', linewidth=2,
            label=f'Mean: {mean_val:.2f}')
plt.axvline(median_val, color='green', linestyle='-', linewidth=2,
            label=f'Median: {median_val:.2f}')
```



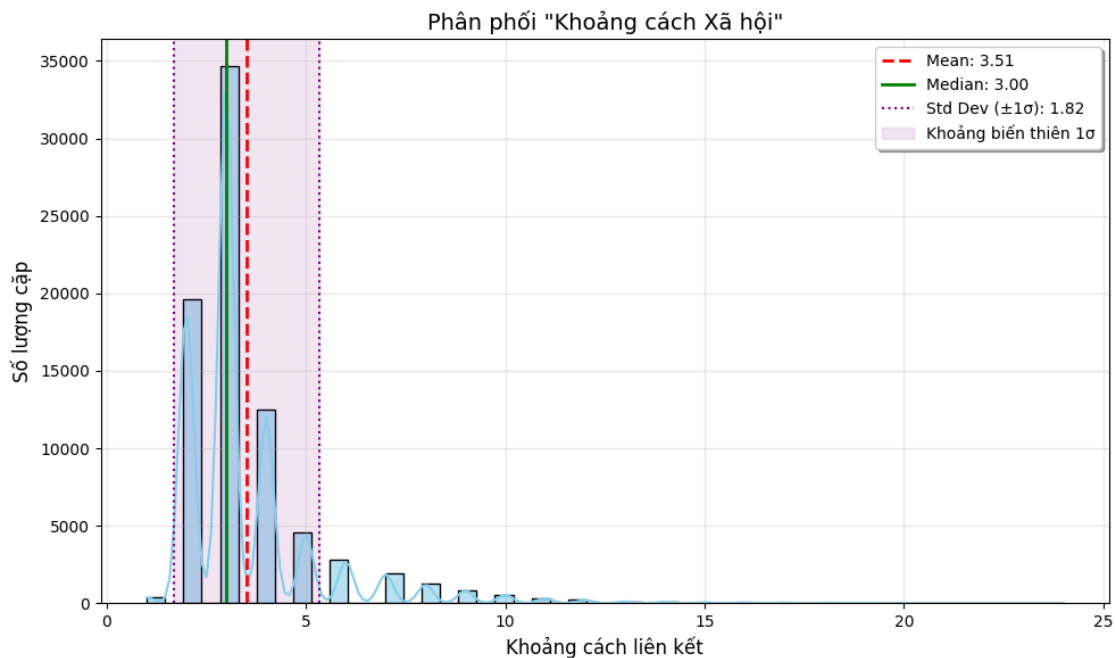
```
plt.axvline(mean_val - std_val, color='purple', linestyle=':', linewidth=1.5,
            label=f'Std Dev ( $\pm 1$ ): {std_val:.2f}')
plt.axvline(mean_val + std_val, color='purple', linestyle=':', linewidth=1.5)
plt.axvspan(mean_val - std_val, mean_val + std_val, color='purple', alpha=0.1,
            label='Khoảng biến thiên 1 ')

plt.title('Phân phối "Khoảng cách Xã hội"', fontsize=14)
plt.xlabel('Khoảng cách liên kết', fontsize=12)
plt.ylabel('Số lượng cặp', fontsize=12)

plt.legend(frameon=True, shadow=True)

plt.grid(True, alpha=0.3)
plt.tight_layout()

plt.show()
```



Từ kết quả cho thấy: Đồ thị lệch phải, với **trung bình là 3.51** và **độ lệch chuẩn là 1.82** cho thấy phần lớn các cặp người dùng có khoảng cách ngắn, chỉ khi một số ít có khoảng cách xa. Điều này cho thấy đồ thị có tính kết nối cao, phù hợp với giả thuyết Sáu bậc phân cách.

Kết luận Do $\bar{d} \leq 6$: Ta chấp nhận giả thuyết H_0 . Mạng lưới tri thức Wikidata thực sự là một “Thế giới nhỏ”, nơi mọi thực thể đều có sợi dây liên kết chặt chẽ.